

1. Software Process: A software process is a set of activities executed to develop software products. Without a repeatable process, only repeatable results you get are errors.

Why follow a defined process? It can be repeated in future projects, evaluated with metrics (cost, quality, time), and improved for better results.

2. Process Improvement — CMM: The Capability Maturity Model (CMM), developed by SEI at Carnegie Mellon University, is a framework for evaluating and improving an organization's software development process. Organizations improve incrementally — not all at once.

CMM — Immature vs Mature Organizations

Immature Organization	Mature Organization
No defined process — everything is ad hoc	Activities are carried out in a planned manner
React to problems instead of preventing them	Processes and products are measured continuously
Cost and schedule estimates are wildly inaccurate	Estimates are accurate due to rigorous measurement
No quality measurement program exists	Processes are continually updated using new tools & experience
Projects frequently overrun cost and time	Standards and policies play key roles in development

CMM Levels: CMM rates an organization on a scale of 1–5. Each level has Key Process Areas (KPAs) that must be achieved to move to the next level.

Level 1 Initial	<i>No process model. Individuals decide everything. No KPAs. Every organization starts here automatically. No planning, Ad hoc decisions, No KPAs defined</i>
Level 2 Repeatable	<i>Basic project management established. Past experience used to manage similar projects.</i> Requirements Management, Software Project Planning, Software Project Tracking & Oversight, Software Subcontract Management, Software Quality Assurance, Software Configuration Management
Level 3 Defined	<i>Organization-wide standard processes defined and documented. V-model integrated.</i> Organization Process Focus, Organization Process Definition, Training Program, Integrated Software Management, Software Product Engineering, Intergroup Coordination, Peer Review
Level 4 Managed	<i>Metrics drive decisions. Quantitative insight into process and product quality.</i> Quantitative Process Management, Software Quality Management
Level 5 Optimizing	<i>Continuous process improvement using data, new techniques and tools.</i> Defect Prevention, Technology Change Management, Process Change Management

3. Models vs Standards vs Guidelines

MODEL	STANDARD
Created by authority as a guideline (better way of doing things)	Created by authority as a definitive way of doing things
Adoption is flexible — levels of adoption vary; not mandatory	Adoption is binary — either successful or failed; mandatory unless exclusion justified
No certification; based on self-assessment	Certification granted after successful audit (e.g. ISO 9001:2008)
May need interpretation; artifacts are optional	Less room for interpretation; less freedom
No periodic assessment; no validity period (e.g. CMMi)	Periodic third-party audits; validity period exists (e.g. ISO 9001:2008)

GUIDELINE	STANDARD
Suggested ways of doing things; not mandatory	Mandatory ways of doing things
More descriptive; may include examples	Less descriptive; examples may not be available
Expressed with 'should', 'may' — can be overruled	Expressed with 'shall', 'must' — overruling is punishable
May or may not be written down	Must be written to avoid miscommunication

4. ISO Software Quality Standards

- **ISO 9126:** Defines quality characteristics (what quality factor means in McCall model terms). Focuses on characteristics visible to users.
- **ISO 9000:2000:** Quality assurance standard with three parts: ISO 9000 — Fundamentals and vocabulary (8 principles), ISO 9001 — Requirements (5 kinds), ISO 9004 — Guidelines for performance improvements

ISO 9000 — 8 Principles

- **Customer Focus:** Satisfy customers' needs and expectations
- **Leadership:** Leaders set direction and communicate it effectively to everyone
- **Involvement of People:** People at all levels are informed, involved, and encouraged to be creative
- **Process Approach:** Managing activities as processes increases efficiency
- **System Approach to Management:** A system is a set of interacting processes
- **Continual Improvement:** Ongoing improvement is a permanent objective
- **Factual Approach to Decision Making:** Decisions based on facts, data, and measured information
- **Mutually Beneficial Supplier Relationships:** Fair terms and contracts with third-party suppliers

ISO 9001 — 5 Kinds of Requirements

- **Systemic Requirements:** A Quality Management System (QMS) with defined quality policy and objectives
- **Management Requirements:** Upper management must support QMS and ensure organization-wide quality awareness
- **Resource Requirements:** Right kind and quantity of resources must back the quality objectives
- **Realization Requirements:** Processes that transform customer requirements into products
- **Remedial Requirements:** Measurement, analysis, and continual improvement of process performance

WEEK 5 — Fundamental Test Process & Test Case Design

1. The Fundamental Test Process (5 Stages)

Stage 1 — Test Planning & Control

- Planning: Decide WHAT will be tested, HOW it will be done, and WHO will do it
- Define Test Completion Criteria — how you know when testing is finished
- Control: Ongoing comparison of progress against the plan; adjust plans if needed
- Planning is not a one-time event — it's continuous throughout testing

Stage 2 — Test Analysis & Design

- Bridge between Planning and Execution
- Identify test conditions (what to test) and combine into test cases (how to test)
- Design test data needed for test conditions
- Predict how the software SHOULD behave — define expected results BEFORE execution

Stage 3 — Test Implementation & Execution

- Running the actual tests — most visible part, but impossible without the other stages
- Set up/tear down the test environment before and after tests
- Run the most important tests FIRST (prioritized during planning)
- Log all test outcomes; compare expected vs actual results
- If discrepancy found → raise a test incident for investigation

Stage 4 — Evaluating Exit Criteria & Reporting

- Check if exit criteria (defined during planning) have been met
- Example: if criterion was 85% statement coverage but only 75% achieved → either change criteria OR run more tests
- Write a test summary report for stakeholders (what was planned, what was achieved, differences)

Stage 5 — Test Closure Activities

- Wrap up the entire testing effort
- Archive test materials and lessons learned for future projects

2. Key Terminologies

Test Condition

An item or event of a component/system that can be verified by one or more test cases — e.g. a function, feature, or structural element. Simply: something in the software you can check.

Test Case

A set of: input values + execution preconditions + expected results + execution post-conditions. It gets the system to a starting state, applies inputs, checks the outcome, and defines the end state.

Test Procedure Specification

A sequence of actions for executing a test (also called a test script). It covers: navigate to screen → input values → check result.

Test Execution Schedule

Orders all test procedures into the correct sequence so the system can be run efficiently.

3. Test Case Design Example

Scenario: An input screen with Title (dropdown), Surname (max 20 alpha + hyphen), First Name (max 20 alpha). All fields required. Press Enter to validate.

Valid Test Case Example: Title: Mr, Surname: "Compo", First Name: "Simmonite" → System moves to job input screen

Invalid Test Case Example: Surname: "123" (contains numbers) → System shows error message

Test Procedure Steps: Navigate to the appropriate input screen, Select a title from the dropdown, Tab to Surname field and enter a value, Tab to First Name field and enter a value, Press the Enter key, Check: does the system move to the next screen (valid) or show an error (invalid)?

WEEK 12 (Session 1) — Test Organization, Approaches & Planning

1. Test Organisation & Independence

Independent testing = testing done by someone OTHER than the developer. It improves effectiveness because authors find it hard to spot their own errors, but others can see them easily.

Levels of Independence (Low → High)

- **Lowest:** The developer tests their own code
- **Low:** Independent testers embedded in the development team
- **Medium:** Independent permanent test team within the organization
- **Medium-High:** Testers provided by operational business units
- **High:** Specialist testers (usability, security, performance)
- **Highest:** Outsourced test team — contractors or other organizations

Benefits & Drawbacks of Independent Testing

Benefits ✓	Drawbacks ⚠
Tester sees different defects than the author	Isolation from the team — dependent on documentation (often outdated)
Tester is unbiased and objective	Tester seen as a bottleneck (last stage, affected by all prior delays)
Tester sees what was ACTUALLY built, not what developer thought was built	Developers lose sense of quality responsibility — they assume testers will find errors
Tester makes no assumptions about quality	Creates an invisible fence between developers and testers — communication suffers

2. Roles — Test Leader vs Tester

Test Leader	Tester
Coordinates test strategy and plan with project managers	Reviews and contributes to test plans
Plans resources, effort, schedule, and cost of testing	Analyses and assesses requirements for testability
Manages test specification, implementation, and execution	Creates test specifications from the test basis
Takes corrective action based on test results and progress	Sets up the test environment
Ensures configuration management of test assets	Prepares test data
Defines metrics for measuring test progress	Executes tests, logs results, documents defects
Decides what to automate and how	Reviews tests written by other testers
Writes test summary report at end of project	Automates tests if required

3. Test Approaches / Strategies

A test approach is how testing will be implemented on a specific project. It can be:

- **Preventative:** Test design starts early in the life cycle to stop defects being built in
- **Reactive:** Testing starts only after design and coding are done (waterfall approach)

Types of Test Approaches

- **Analytical (Risk-based):** Directs testing to areas of greatest risk
- **Model-based:** Uses statistical info about failure rates (reliability growth models)
- **Methodical:** Failure-based (error guessing, fault attacks), checklist-based, quality characteristic-based
- **Standard-compliant:** Based on industry standards e.g. Railway Signalling standards, MISRA
- **Process-compliant:** Follows agile methodologies or traditional waterfall processes
- **Dynamic/Heuristic (Exploratory):** Testing is reactive; execution and evaluation happen simultaneously
- **Consultative:** Coverage driven by advice from technology/business domain experts
- **Regression-averse:** Reuses existing tests; extensive automation of regression tests

4. Test Planning

Test planning is the most important activity of a test leader. It defines the shape and size of the test effort and produces a master test plan / project test plan.

Test Plans in the V-Model

The Master (Project) Test Plan sits at the top. It spawns separate test-level plans as more detail becomes known:

Plan Level	Description
Master / Project Test Plan	High-level plan produced early in the project (e.g. initiation phase). Covers all test activities. Contains enough info to establish the test project.

Component Test Plan	Detailed plan for unit/component-level testing of individual code modules.
Integration Test Plan	Detailed plan for testing interfaces and interactions between components.
System Test Plan	Detailed plan for end-to-end system-level testing.
Acceptance Test Plan	Detailed plan for verifying the system meets customer requirements; often involves customer/user.

Note: More information, change requests and changing priorities require an iterative process. This usually results in several versions of test plans being produced, reviewed and re-issued.

Factors Influencing the Test Planning Process

8 key factors that shape how a test plan is developed:

Risks	Constraints	Resources	Criticality
Test Policy	Scope of Testing	Test Objectives	Testability

IEEE 829 Test Plan Sections (Detailed)

The IEEE 829 standard defines 16 sections for a test plan. Here are the key sections with their content:

#	Heading	Details
1	Test Plan Identifier	A unique identifying reference such as 'Doc ref XYZ v2'
2	Introduction	A brief introduction to the document and the project for which it has been produced
3	Test Items	A test item is a software item that is the object of testing. A software item = source code, object code, job control code, or control data. Should include documentation references (e.g. design docs).
4	Features to be Tested	A feature is a distinguishing characteristic of a software item (e.g. performance, portability, functionality). Identify ALL features and feature combinations with associated test design specs.
5	Features NOT to be Tested	Identify all features and significant combinations NOT being tested and state the reasons for exclusion.
6	Approach	The overall test strategy: techniques, coverage measures, types of testing to apply.
7	Item Pass/Fail Criteria	What determines if a test item has passed or failed.
8	Suspension & Resumption Requirements	Conditions under which testing is suspended and what must happen before it can resume.
9	Test Deliverables	All documents, reports, and artifacts to be produced (test plans, specs, logs, reports).
10	Testing Tasks	All tasks required to plan and execute the tests.
11	Environment Needs	Hardware, software, tools, network, and other requirements for testing.
12	Responsibilities	Who is responsible for each testing task.
13	Staffing & Training Needs	Skills needed; training required for team members.
14	Schedule	Timeline for all testing tasks and milestones.
15	Risks & Contingencies	Potential risks to the test project and plans to handle them.
16	Approvals	Who must approve the test plan and space for signatures.

IEEE 829 Test Plan — SPACEDIRT Acronym

- **S — Scope:** Test items, features to be tested, features NOT to be tested
- **P — People:** Responsibilities, staff, training needs, approvals
- **A — Approach:** How testing will be carried out
- **C — Criteria:** Item pass/fail criteria, suspension and resumption requirements
- **E — Environment Needs:** Hardware, software, network, test tools required
- **D — Deliverables:** What test documents/outputs will be produced
- **I — Identifier & Introduction:** Document ID and scope introduction
- **R — Risks & Contingencies:** What could go wrong and how to handle it
- **T — Tasks & Schedule:** All testing tasks and their timelines

Entry Criteria (When can testing START?): Test environment is available and working. Test tools are installed and ready. Testable code is available. All test data is available and correct. All test design activity has completed

Exit Criteria (When does testing STOP?): All planned tests have been run. A certain level of requirements coverage has been achieved. No high-priority or severe defects remain outstanding. All high-risk areas have been fully tested. Budget has been spent. Schedule date reached (e.g. mandatory release)

WEEK 12 (Session 2) — Test Estimation, Monitoring & Control

1. Test Estimation

Two main approaches to estimating how much time and effort testing will require:

Metrics-Based Approach	Expert-Based Approach
Based on data collected from previous/similar projects	Based on the experience and judgment of experts (also called Wide Band Delphi)
Metrics include: number of test cases, time to develop/run tests, defects found, environment outages	Experts include: business experts, developers, architects, testers, analysts
More objective and data-driven	More subjective but useful when historical data is not available
Actual costs must be recorded for reuse in future estimates	Can combine expert views then meet to agree on a final estimate
Most accurate when data is from truly similar projects	Works by distributing specs to task owners → individual estimates → amalgamate

2. Factors Affecting Test Effort

Product Characteristics: Size of the test basis (requirements, specs). Complexity of the final product. Amount of non-functional requirements. Security requirements (e.g. BS 7799). Documentation requirements (legislation may demand more). Availability and quality of the test basis

Development Process Characteristics: Timescales and deadlines. Budget available. Skill levels of developers and testers (lower skill → more defects & more documentation needed). Amount of automated testing

Expected Outcome: Expected number of errors/defects. Number of test cases to be written

3. Test Progress Monitoring

After test planning, activities must be constantly reviewed against what is actually happening. Purpose: provide feedback and visibility of test progress.

Common Test Metrics: % of test cases prepared (vs planned). % of test environment preparation done. Test cases run / not run / passed / failed. Defect information: density, found/fixed, failure rate, retest results. Test coverage of requirements, risks, or code. Tester confidence levels in the product. Test milestone dates. Testing costs vs benefit of finding the next defect

Trend: Dashboards reflect all relevant metrics on one screen for maximum visibility and impact.

4. Test Reporting (IEEE 829 Test Summary Report)

Test reporting summarises what was tested, what happened, and what should happen next. The IEEE 829 standard defines 8 sections:

#	Heading	Details
1	Test Summary Report Identifier	The specific identifier allocated to this document, e.g. TSR XYZ v1
2	Summary	Identifies the items tested (including version numbers). Documents the environments in which the test activity took place. References the testing documentation for each test item, e.g. test plan, test cases, test defect reports.
3	Variances	Reports deviations from the test approach or strategy, test plan, test specification or test procedures.
4	Comprehensive Assessment	Measures the actual progress made against the exit criteria and explains why any differences have arisen.
5	Summary Results	Provides an overview of the results from the test activities; includes details of defects raised and fixed, as well as those that remain unresolved.
6	Evaluation	Provides an evaluation of the quality of each test item, including a view of the risks of failure in production of these test items. Based upon the test result metrics and test item pass/fail criteria.
7	Summary of Activities	A summary of the major test activities and events such as test environment unavailability, success or weaknesses of the test specification process, etc. Should also include resource usage data, e.g. planned spend against actual spend.
8	Approvals	Identifies all approvers of the document.

5. Test Control

Test control uses monitoring data to keep testing on track. Required especially when testing is behind schedule.

Examples of Test Control Actions: Reprioritize tests when a risk occurs (e.g. software delivered late). Change the test schedule due to environment unavailability. Set entry criterion requiring defect fixes to be retested by developer before acceptance. Adjust risk ratings after reviewing product risks. Adjust test scope to handle late change requests

WEEK 14 — Testing Maturity Model (TMM)

The TMM is a model specifically for evaluating and improving the testing process (unlike CMM which covers the entire software development process). It has 5 levels — same structure as CMM but focused on testing.

TMM Structure: Each level (except Level 1) has Maturity Goals → Maturity Sub-goals → Activities, Tasks & Responsibilities (ATRs)

Level 1 Initial	<i>Testing is done in an ad hoc, unplanned manner. No goals specified at this level.</i> Testing begins only after code is written. Test cases designed and executed in ad hoc manner. No trained testers, tools, or environments. Testing seen only as proof that the system works, not as a quality activity
Level 2 Phase Definition	<i>Testing separated from debugging; basic planning and techniques introduced.</i> • Goal 1: Develop Testing & Debugging Goals — committees formed; goals documented; tracked from discovery to fix • Goal 2: Initiate Test Planning — identify objectives, analyze risks, devise strategies, develop test specs, allocate resources • Goal 3: Institutionalize Basic Testing Techniques — expert group recommends techniques and tools; management enforces consistent use

Level 3 Integration	<i>Testing fully integrated with the development process from the start.</i> - Goal 1: Establish a Software Test Group — independent test team with authority to influence quality - Goal 2: Establish Technical Training Program — ongoing training in testing techniques, tools, and review processes - Goal 3: Integrate Testing into Software Life Cycle — V-model adopted; test phases mapped to dev phases - Goal 4: Control and Monitor Testing Process — track effort, cost, and schedule; define metrics and contingency plans
Level 4 Management & Measurement	<i>Testing broadened with formal review programs and measurement programs.</i> - Goal 1: Establish Organization-wide Review Program — detect defects early with low cost; review policies defined - Goal 2: Establish Test Measurement Program — define test metrics, data collection plan, improvement action plan - Goal 3: Evaluate Software Quality — define quality attributes/goals; collect metrics to support quality goals
Level 5 Optimization, Defect Prevention & Quality Control	<i>Continuous improvement using data, statistics, and advanced tools.</i> - Goal 1: Application of Process Data for Defect Prevention — root cause analysis, defect prevention team, eliminate recurring defects - Goal 2: Statistical Quality Control — measurable quality goals, Pareto analysis, histograms, control charts, usage modelling - Goal 3: Test Process Optimization — identify and improve testing practices; evaluate new tools and technologies continuously

CMM vs TMM — Quick Comparison

CMM	TMM
Covers the entire software development process	Covers the testing process specifically
Level 1–5 (Initial to Optimizing)	Level 1–5 (Initial to Optimization)
Developed by SEI at Carnegie Mellon	Developed recognizing testing as a distinct process
Used to rate overall organizational maturity	Used specifically to improve test process maturity
KPAs: e.g. Requirements Management, SQA	Goals: e.g. Test Planning, Test Group, Test Measurement

WEEK 15 — Static Testing

1. What is Static Testing? Static testing is a human testing technique that does NOT involve executing or running the program. Instead, it involves checking/monitoring software at each phase of the SDLC without executing code.

Why is Static Testing Important?

- Identify Defects Early:** Fix issues before deployment — saves time and money
- Improve Code Quality:** Catch syntax errors, coding standard violations, and design flaws
- Reduce Maintenance Costs:** Cheaper to fix defects found early vs late in development

2. Types of Static Testing

- Reviews:** Examining software or documentation to identify defects. Includes formal reviews, informal reviews, walkthroughs, inspections.
- Static Analysis:** Analyzing code without executing it using tools (SonarQube, PMD, FindBugs) to find defects, vulnerabilities, performance issues.
- Code Inspection:** Manual review of source code by a team of developers to find defects, violations, and issues.

- **Document Review:** Reviewing documentation for completeness, accuracy, and currency — done by technical writers or subject matter experts.
- **Modeling:** Creating models (data flow diagrams, state diagrams, use case diagrams) to identify defects or inconsistencies.

3. Static Testing Techniques

A. Software Inspection (Formal — Most Rigorous)

Applied to specific parts of the product (SRS, code, design) in the EARLY stages of SDLC. Has 6 phases:

- **Phase 1: Planning for Inspection Meeting:** Identify product to inspect; assign a Moderator who checks readiness, selects team, assigns roles, schedules meeting, distributes material.
- **Phase 2: Overview:** Author presents logic, reasoning, functions, and purpose of the product to the team. Everyone is made familiar with the objectives.
- **Phase 3: Individual Preparation:** Each member studies material independently, identifies potential errors, records them in a log, and submits to moderator. Moderator compiles logs and sends to author.
- **Phase 4: Inspection Meeting:** Author discusses issues raised. Team decides if each issue is a real error. Moderator concludes with a summary list of errors for the author to fix.
- **Phase 5: Rework:** Author fixes all bugs from the summary list and reports back to moderator.
- **Phase 6: Follow-up:** Moderator checks if all errors are resolved. If yes → document released. If no → unresolved issues added to report and another meeting scheduled.

B. Structured Walkthroughs (Informal — Simpler)

Less formal and less rigorous than inspection. Has 4 steps:

- **Step 1: Organization:** Assign roles — Coordinator (organizes), Presenter (introduces item), Scribe (notes issues), Tester (finds defects), Maintenance Oracle (future maintenance focus), Standards Bearer (checks compliance), User Representative (user needs).
- **Step 2: Preparation:** Prepare for walkthrough, think of basic test cases needed to test the product.
- **Step 3: Walkthrough:** Tester brings a small set of test cases to the meeting; executes them mentally; records results on paper or presentation.
- **Step 4: Rework & Follow-up:** Bugs resolved; if no bugs found → product approved for release; otherwise → another walkthrough conducted.

C. Technical Reviews (Higher-Level — Involves Management)

Assesses product conformance to standards, guidelines, and specifications. Higher-level than inspection or walkthrough. Process:

- Moderator gathers and distributes material to all team members
- Moderator prepares indicators to evaluate against specifications and standards: consistency, documentation, adherence to standards, completeness, problem definition
- Results recorded in a document with defects AND suggestions
- Defects are resolved and suggestions improve the product

Inspection	Walkthrough
Formal and rigorous	Informal and simpler
6 phases with defined process	4 steps
Inspector independently logs errors before meeting	Tester mentally executes test cases during meeting
Moderator manages the entire process	Coordinator manages the team
Used for SRS, code, design in early SDLC	Used for any software artifact
Higher overhead	Lower overhead; good for early feedback

4. Static Testing Tools

- **Code Review Tools:** Code Collaborator, Crucible, Gerrit — review source code for defects/vulnerabilities
- **Syntax Checkers:** JSLint, ESLint, PyLint — check for syntax errors and coding standard violations
- **Bug Tracking Tools:** Bugzilla, JIRA, Mantis — track and manage defects found during testing

- **Static Analysis Tools:** SonarQube, PMD, FindBugs — analyze source code for defects, vulnerabilities, performance issues
- **Automated Documentation Generators:** Doxygen, NaturalDocs — generate documentation from source code
- **Model Checking Tools:** Alloy, NuSMV — check models for defects or inconsistencies

WEEK 13 — Incident Management & Configuration Management

1. Incident Management An incident is any unplanned event occurring that requires further investigation. In testing, this means anything where the actual result differs from the expected result. When investigated, an incident may turn out to be a defect, a change to a specification, or an issue with the test itself. Incidents can be raised at any point in the SDLC, from reviews of the test basis through to test specification and execution.

IEEE 1044 Definition

According to IEEE 1044 (Standard Classification for Software Anomalies), incident management is the process of recognising, investigating, taking action and disposing of incidents. It ensures incidents are tracked from recognition through correction, retest and closure. Organisations should document their incident management process and appoint an incident manager/coordinator to oversee it.

Objectives of Incident Reports

- **Developer Feedback:** Give developers enough information to identify, isolate and correct the problem, since they usually weren't present when it was found — the more detail, the better.
- **Progress Tracking:** Give test leaders a way to track quality and testing progress, using metrics like number of incidents raised, their priority, and whether they've been corrected and signed off.
- **Process Improvement:** Document the point of injection for each incident (e.g., a defect in requirements or code) so process improvement can target that area and prevent recurrence.

Details Typically Included in an Incident Report

- **Identification & Status:** Date of issue, issuing organisation, author, approvals and status.
- **Severity & Impact:** Scope, severity and priority of the incident, plus its degree of impact on stakeholders and the system.
- **References & Environment:** Identity of the test case specification that revealed the problem, the test item/configuration item, environment, and the SDLC phase in which it was observed.
- **Results:** Expected and actual results, and the date the incident was discovered.
- **Description:** Enough detail to reproduce and resolve the incident, including logs, database dumps or screenshots.
- **Resolution:** Conclusions, recommendations, approvals, related global issues, and the change history of actions taken to isolate, repair and confirm the fix.

Test Incident Report Outline

- **Test Incident Report Identifier:** A unique reference for the report.
- **Summary:** A brief summary of the incident.
- **Incident Description:** Test items, environment, description of what happened (expected vs. actual), procedure followed, attempts to repeat, testers/observers present, and date/time.
- **Impact:** The effect the incident has on testing, stakeholders and the project.

2. Configuration Management Configuration management is about clearly identifying the items that make up a software system — source code, test scripts, third-party software, hardware, data, and both development and test documentation — and managing them carefully, thoroughly and attentively throughout the project and product life cycle.

Why It Matters for Testing

- **Testware Management:** Lets testers manage their testware and test results using the same configuration management mechanisms as source code and documentation, since testware is just as valuable.
- **Build Support:** Supports the build process needed to deliver a test release into the test environment — simply emailing zip archives isn't reliable, since they can get polluted or contain left-over previous versions.
- **Traceability:** Maps what is being tested to the underlying files and components, so defects can be reported against a version-controlled baseline. Ideally, testers receive an organised, version-controlled release accompanied by a test item transmittal report or release notes, per IEEE 829 guidelines.

Planning for Configuration Management

Configuration management is as complex as test environment management, so advance planning is critical. Procedures and tools should be selected during project planning, and as the project proceeds, the configuration process and key interfaces to the rest of development should be documented. This helps the team avoid surprises at test execution time — like testing the wrong software, receiving uninstallable builds, or reporting irreproducible defects against versions of code that don't actually exist in the test environment.